Task – 5.2c

Name – Nirosh Ravindran

1. It provides a secure gateway for accessing a private network from an external network. It mitigates unauthorized access and potential attacks on resources within the private network.

   In case if we want to use a Linux instance in a private network via SSH from an external network, we can use bastion host to control and secure these connections.

   Bastion hosts mainly run on public subnet on an EC2 instance. When we want to access the Linux instance, we should setup the instance in a way where the SSH traffic is first directed towards the bastion host first and then reaches the Linux instance. This will add an extra layer of security to the

2. It provides a static and public IPv4 address that is designed for dynamic cloud computing. It associates the address with any instance or network interface in our account's VPC.

   An instance's address can be switched quickly if one fails. We will be able to move all attributes of the network interface from one instance to another in a single step when we associate the elastic IP with the network interface instead of directly with an instance.

   This helps in managing instances and maintaining uninterrupted service.

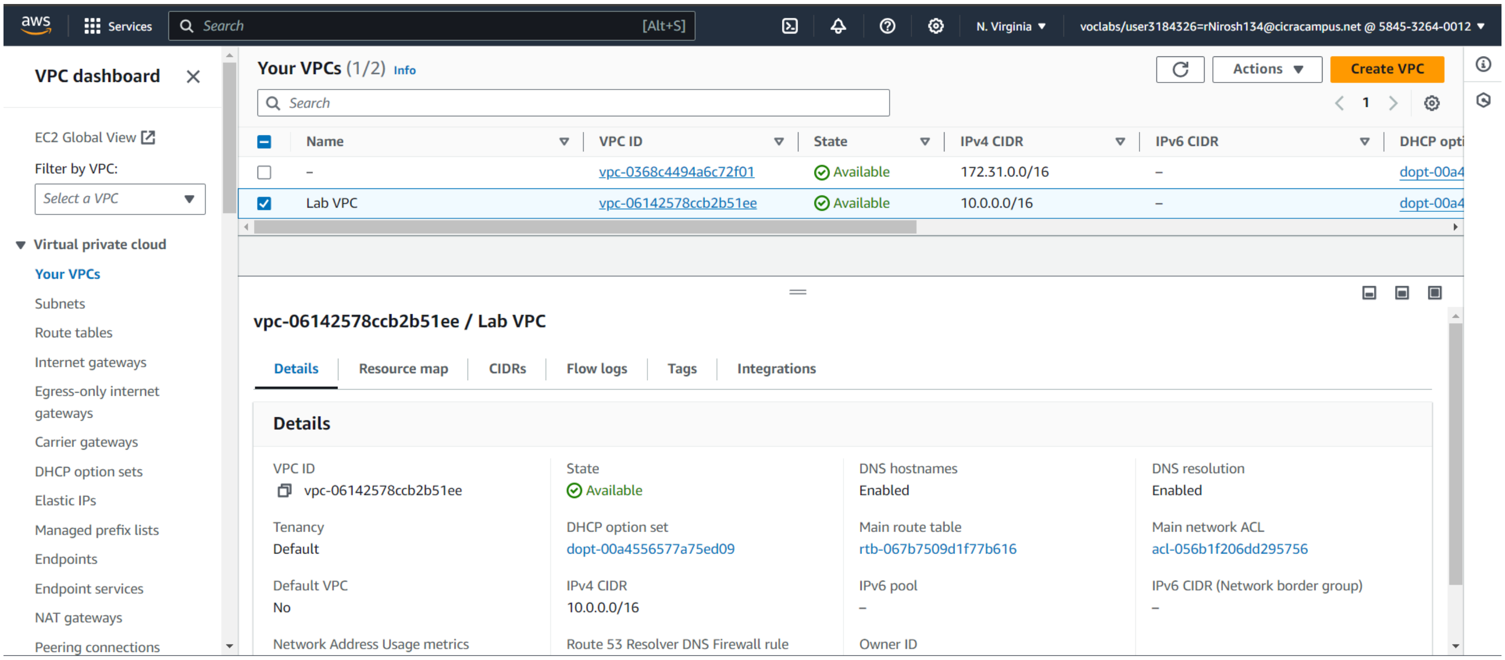
3. SSH pass through is needed to securely access resources within a private network or subnet from an external network or a local computer. Using this, we establish a secure connection to a public host like bastion host to reach the private resources.

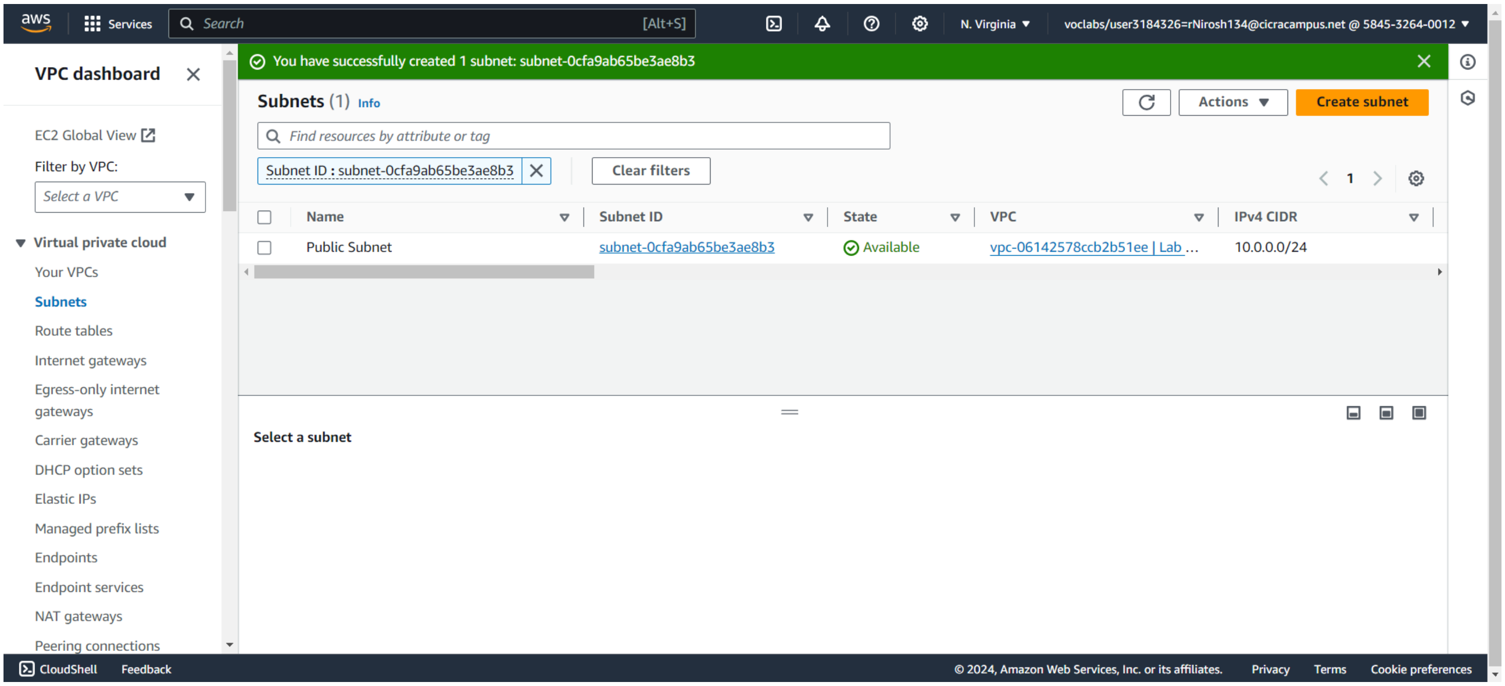
   When data is being in transit over an untrusted network, SSH will ensure data remains protected by encrypting the communication between the host and target. This will also provide a controlled access point, reducing the attack surface and enhancing overall security posture.

Module 6 - Challenge Lab: Creating a VPC Networking Environment for the Café

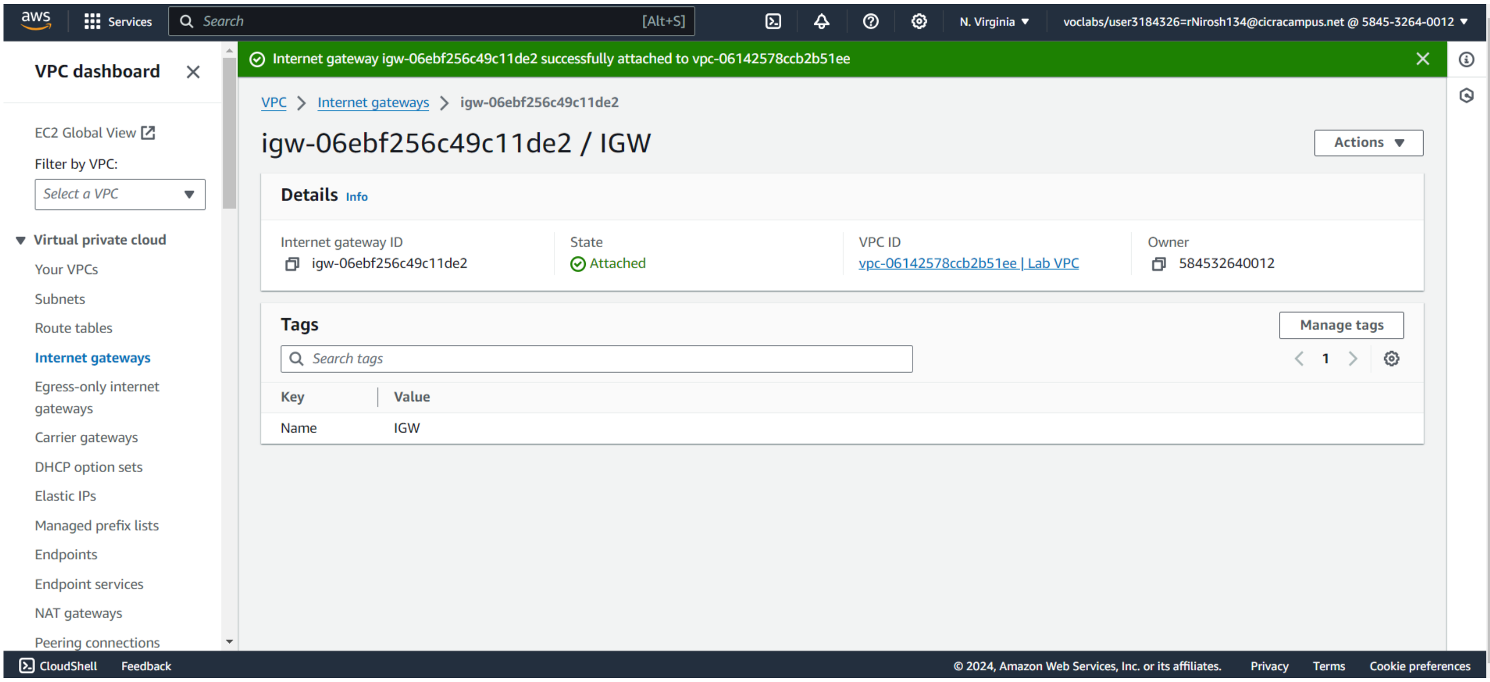
Task 1: Creating a public subnet

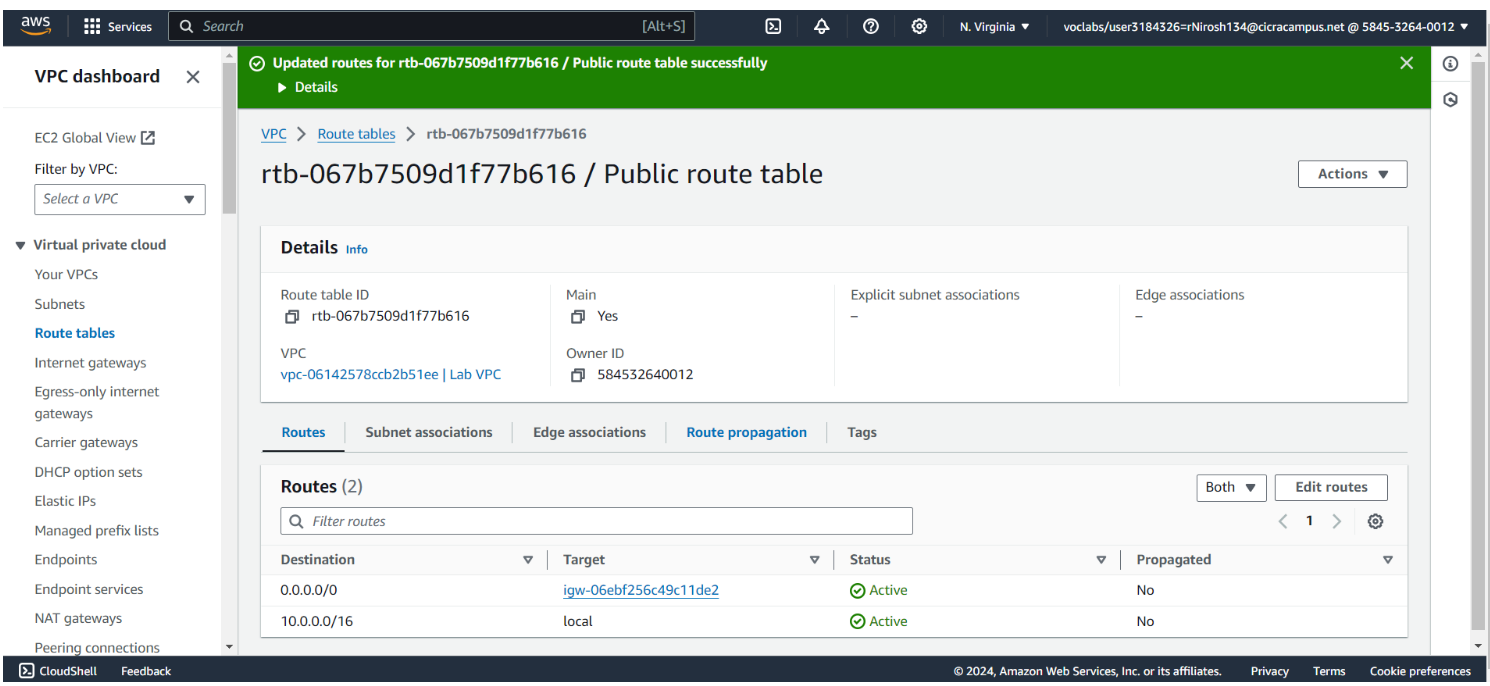
We're setting up a public subnet in our Lab VPC to connect to the internet. First, we create the subnet with specific settings like its name and IP address range.





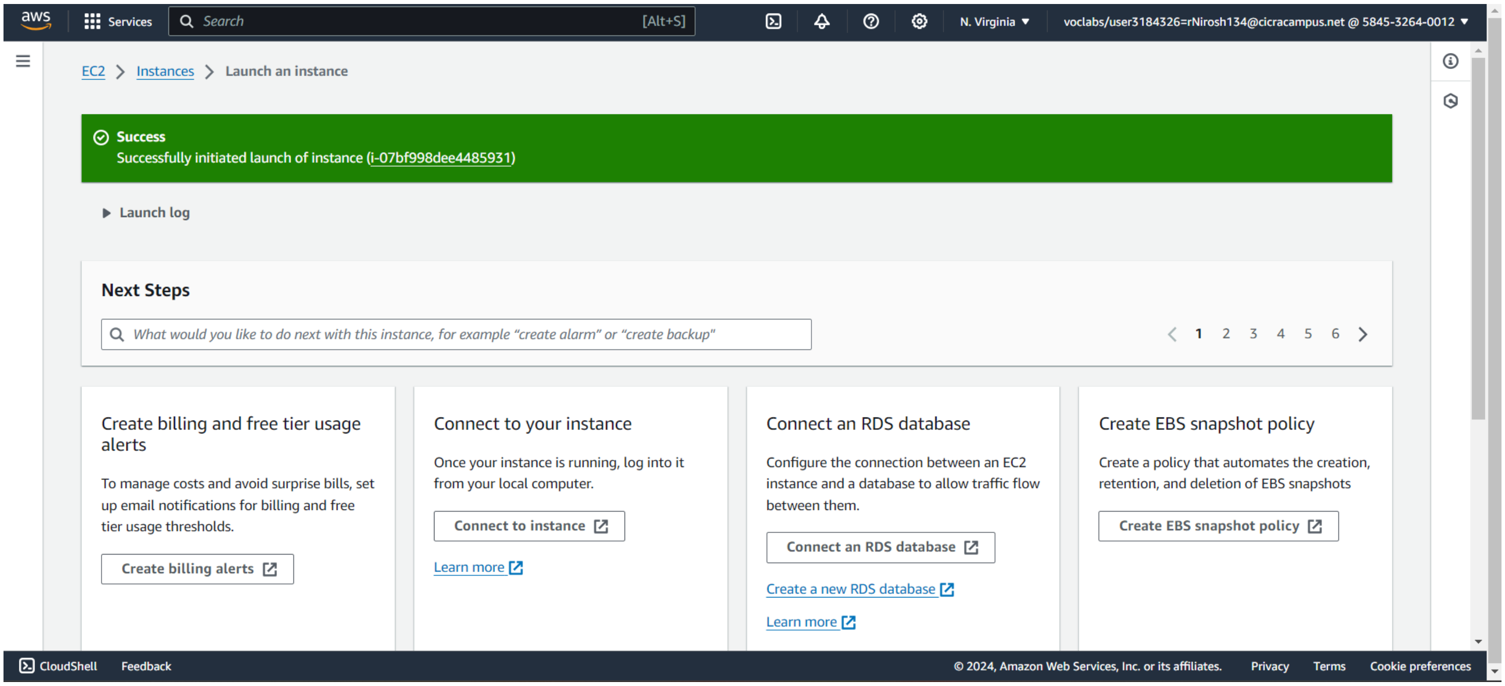
Then, we add an internet gateway to our VPC to enable internet access for the subnet. Finally, we update the route table to direct internet-bound traffic through the gateway.



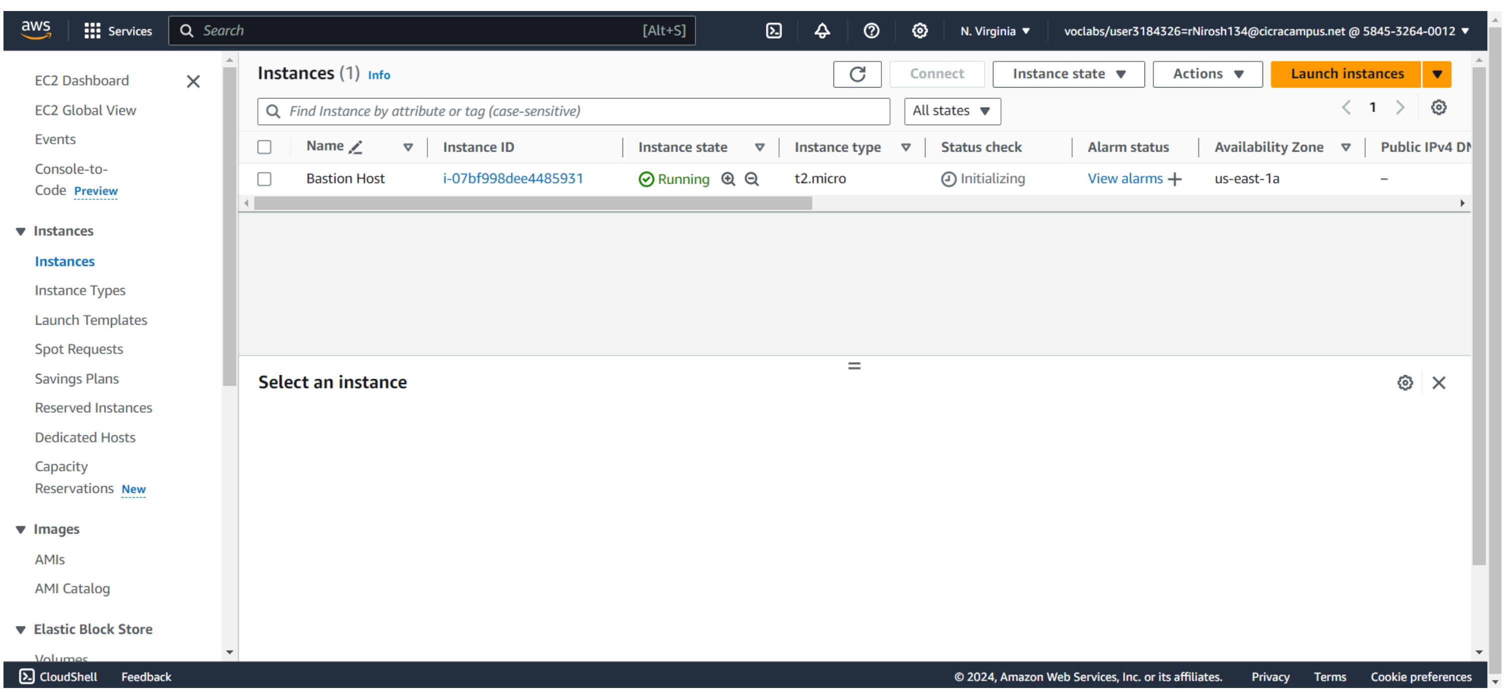


Task 2: Creating a bastion host

We're creating a bastion host in the Public Subnet of our Lab VPC using Amazon EC2. This host acts as a secure gateway for accessing instances in private subnets.

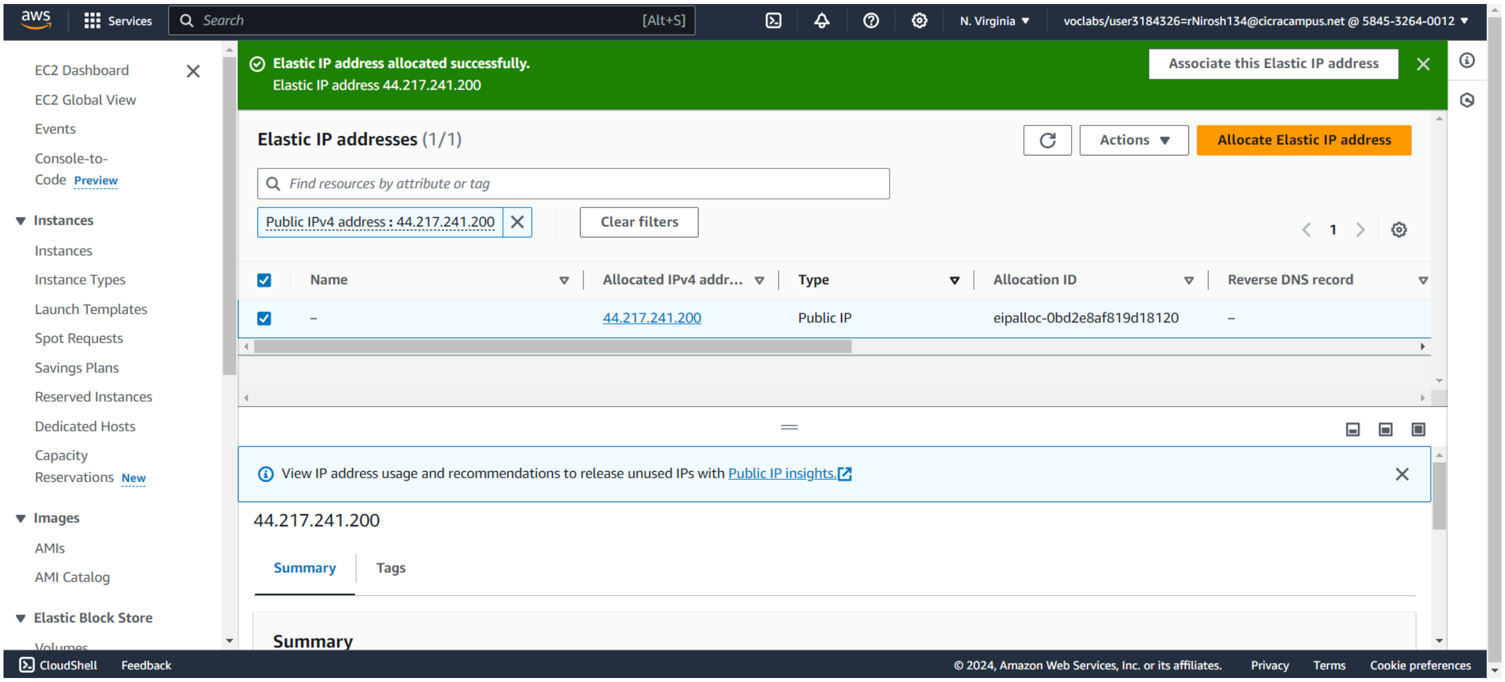


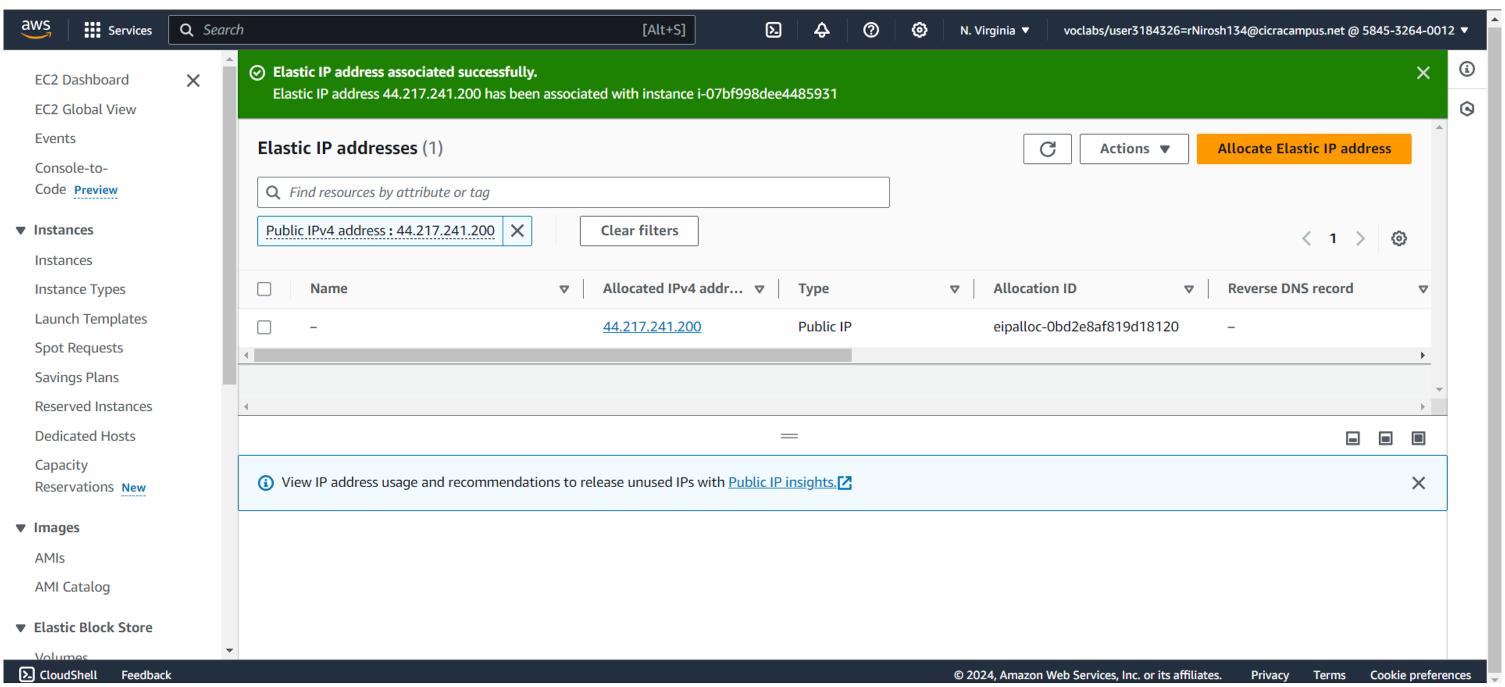
We then choose specific settings for the instance, like its type and security group, to ensure only authorized access.

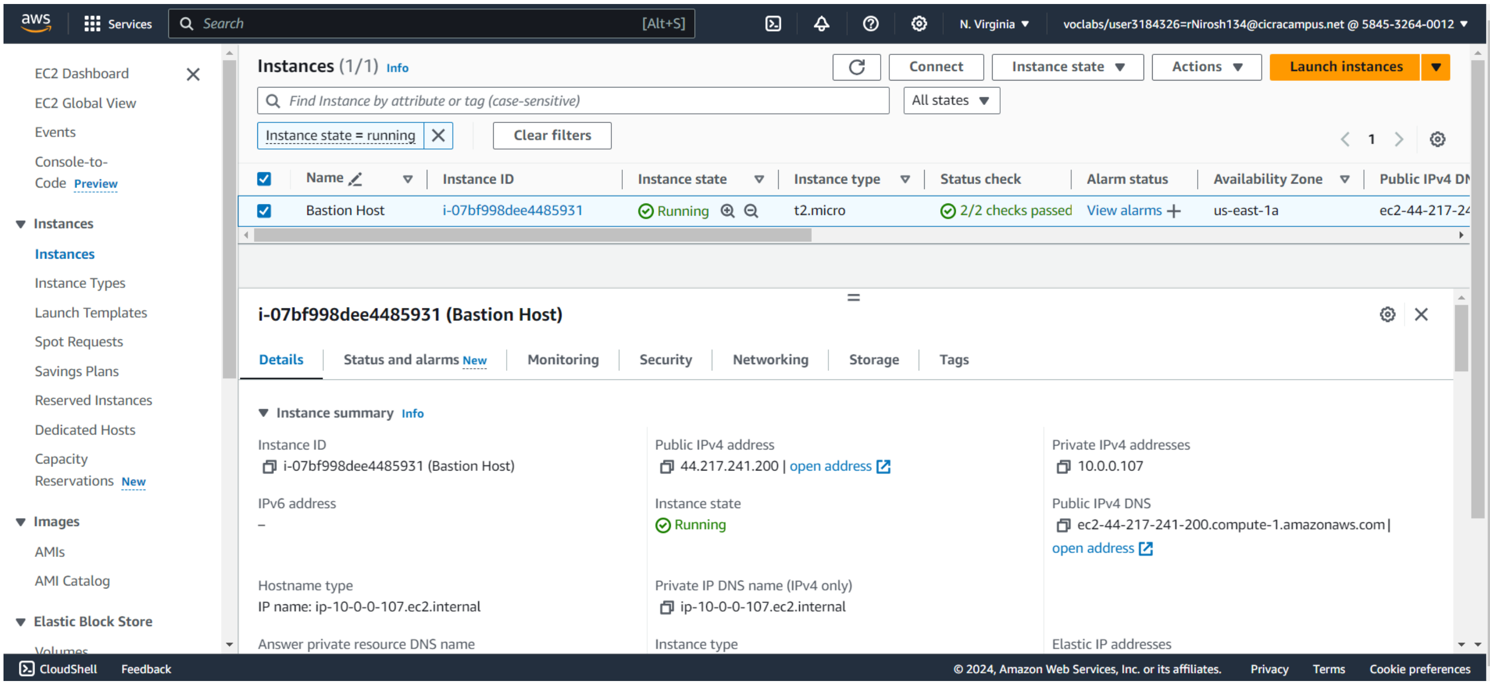


Task 3: Allocating an Elastic IP address for the bastion host

In this task, we're giving our bastion host a special kind of address called an Elastic IP (EIP). Right now, the bastion host can't be accessed from the internet because it doesn't have a public address.



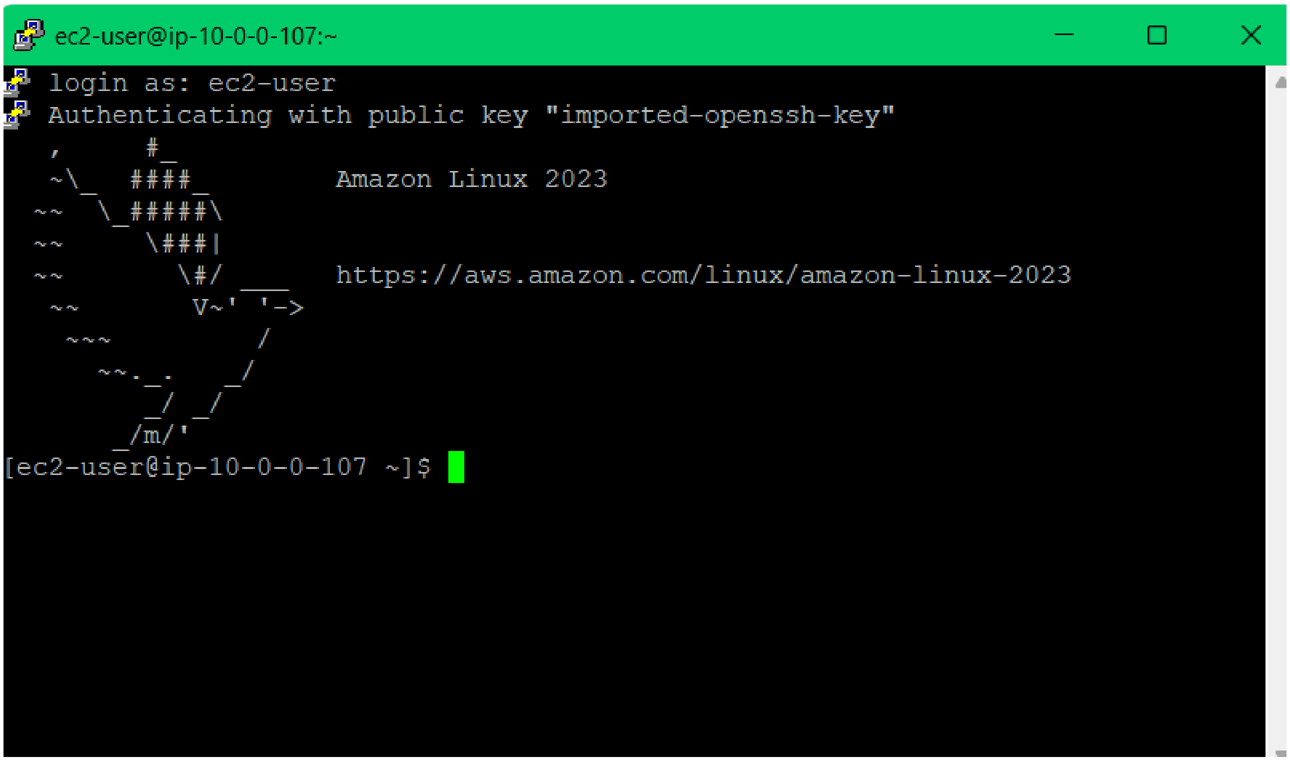



aws
Services
Search
[Alt+S]
N. Virginia
voclabs/user3184326=rNirosh134@cicracampus.net @ 5845-3264-0012
EC2 Dashboard
EC2 Global View
Events
Console-to-Code Preview
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Instances (1/1) Info
Connect
Instance state
Actions
Launch instances
Find Instance by attribute or tag (case-sensitive)
All states
Instance state = running
Clear filters
1
Name
Instance ID
Instance state
Instance type
Status check
Alarm status
Availability Zone
Public IPv4 DN
Bastion Host
i-07bf998dee4485931
Running
t2.micro
2/2 checks passed
View alarms +
us-east-1a
ec2-44-217-24
i-07bf998dee4485931 (Bastion Host)
Details
Status and alarms New
Monitoring
Security
Networking
Storage
Tags
Instance summary Info
Instance ID
i-07bf998dee4485931 (Bastion Host)
Public IPv4 address
44.217.241.200 | open address
Private IPv4 addresses
10.0.0.107
IPv6 address
–
Instance state
Running
Public IPv4 DNS
ec2-44-217-241-200.compute-1.amazonaws.com | open address
Hostname type
IP name: ip-10-0-0-107.ec2.internal
Private IP DNS name (IPv4 only)
ip-10-0-0-107.ec2.internal
Answer private resource DNS name
Instance type
Elastic IP addresses
CloudShell
Feedback
© 2024, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Task 4: Testing the connection to the bastion host

In this task, we'll use a special file (SSH key) to test our connection to the bastion host. We should download the PPK file from the instructions.

After that, this file will be used to connect to your bastion host using SSH. Then we will test the connection using the putty terminal



Task 5: Creating a private subnet

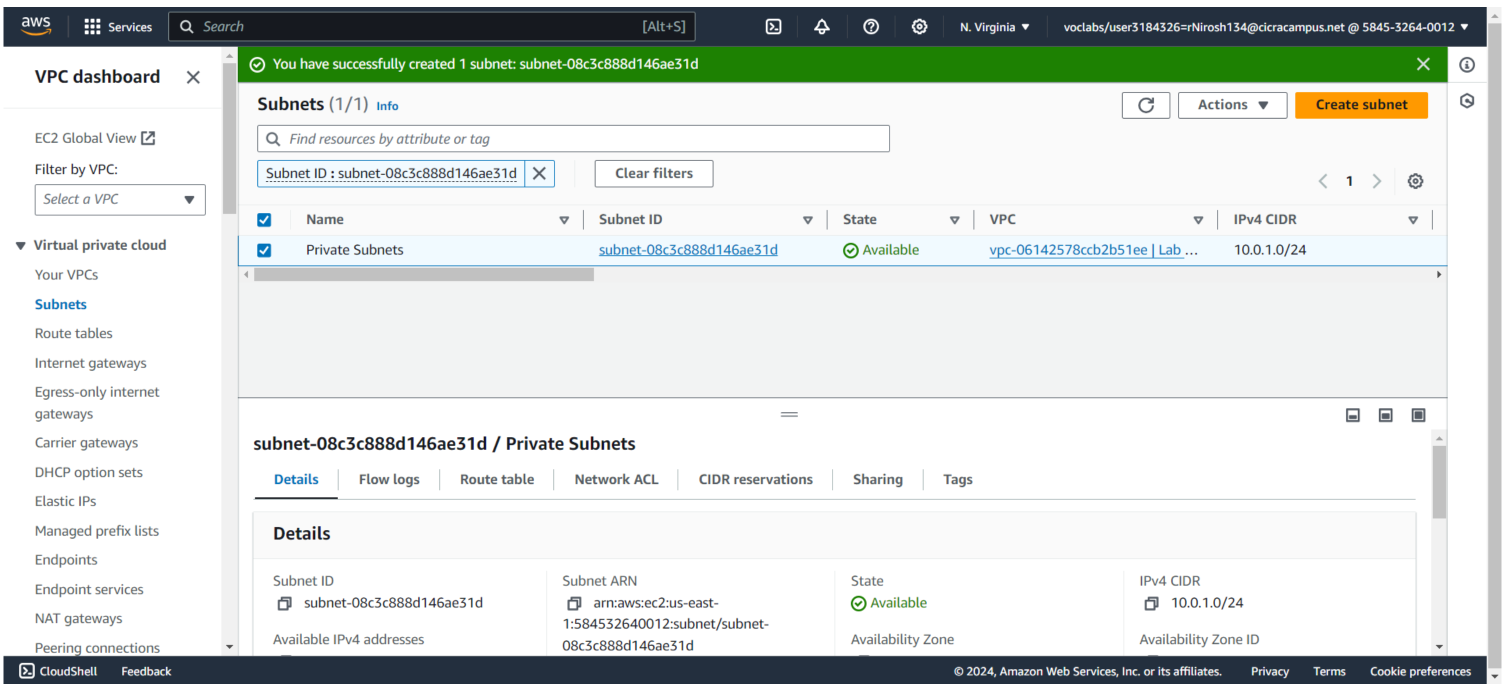
In this task, we're making a private subnet within our Lab VPC.

To do this, we create a private subnet in the console with specific settings:

We name it 'Private Subnet'.

It's in the same availability zone as the public subnet.

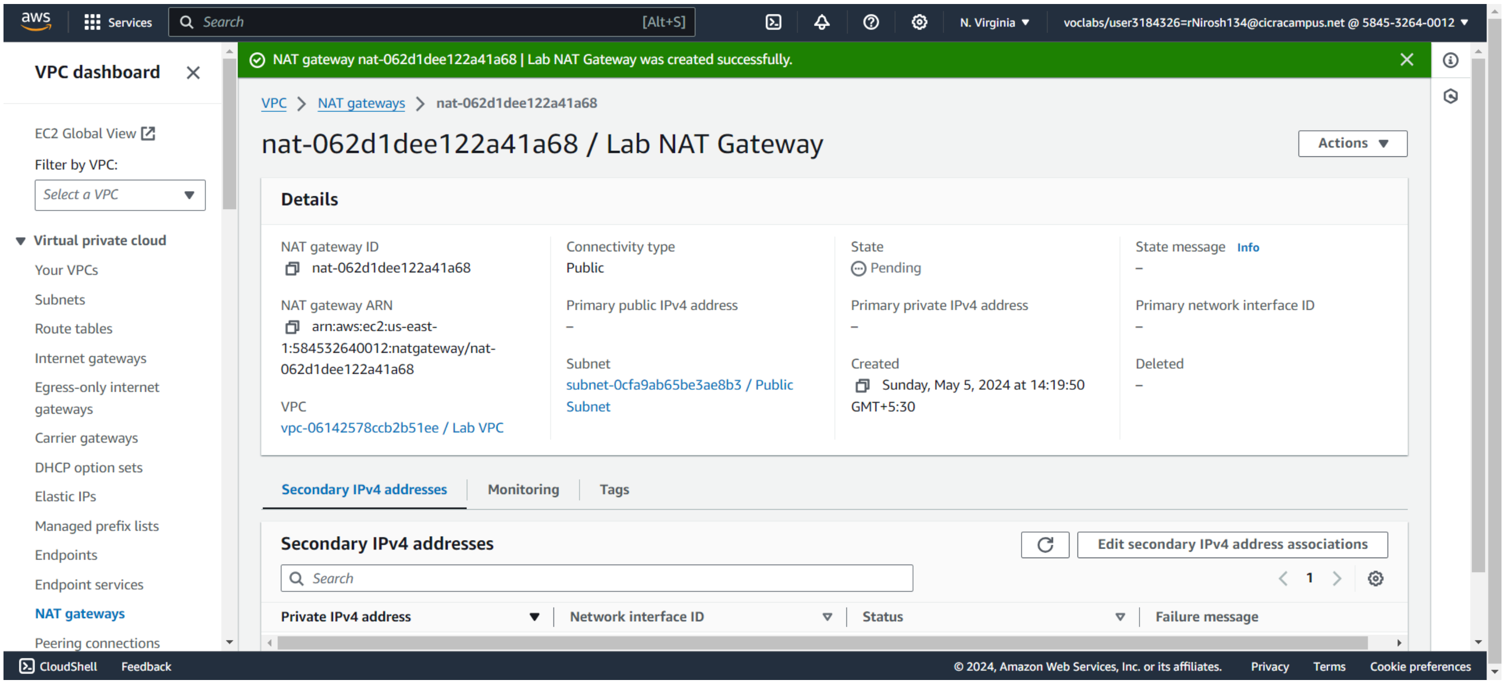
The IP address range for this subnet is 10.0.1.0/24.



Task 6: Creating a NAT gateway

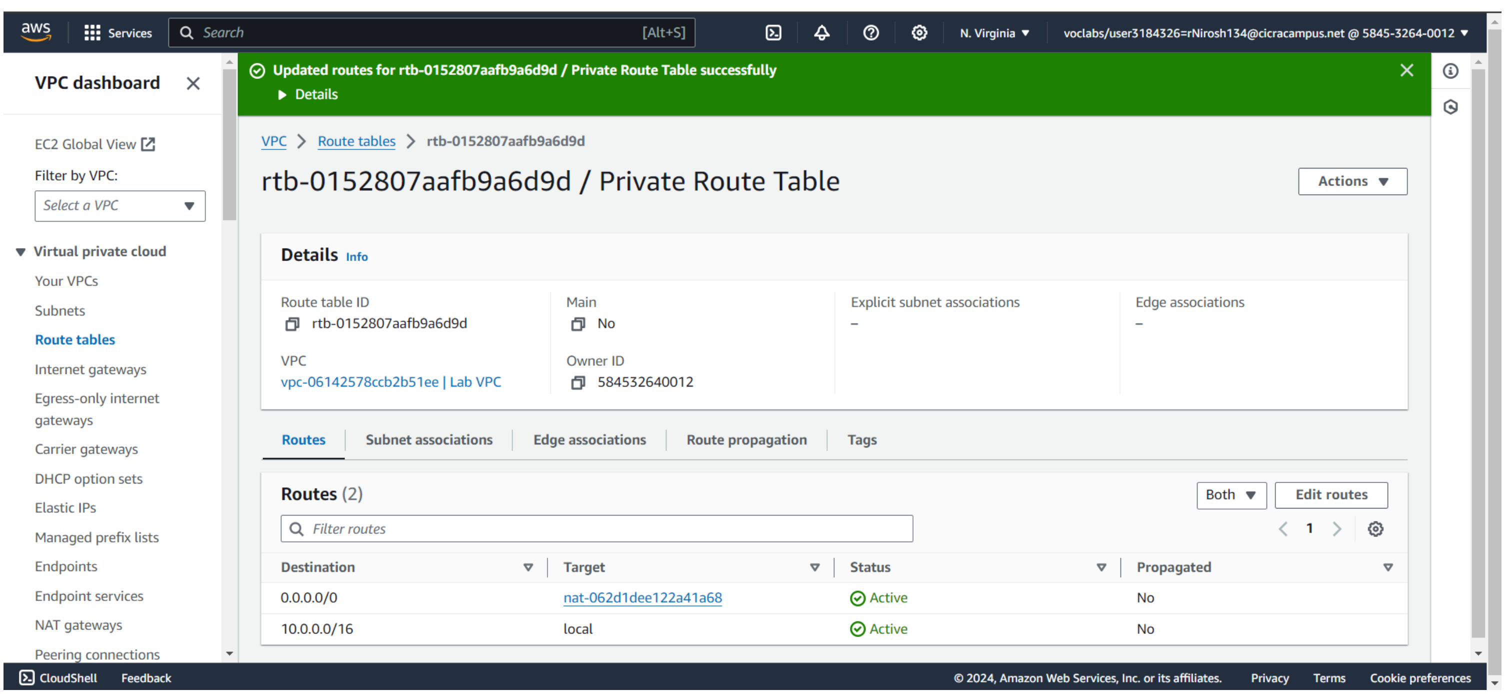
In this task, we're setting up a way for resources in our Private Subnet to access the internet.

First, we create a NAT gateway named 'Lab NAT Gateway' in our Public Subnet. This gateway needs an Elastic IP address.



Next, we create a new route table named 'Private Route Table' that directs all internet-bound traffic (0.0.0.0/0) to the NAT gateway.

Finally, we attach this route table to the Private Subnet we created earlier. This ensures that resources in the Private Subnet can reach the internet through the NAT gateway.

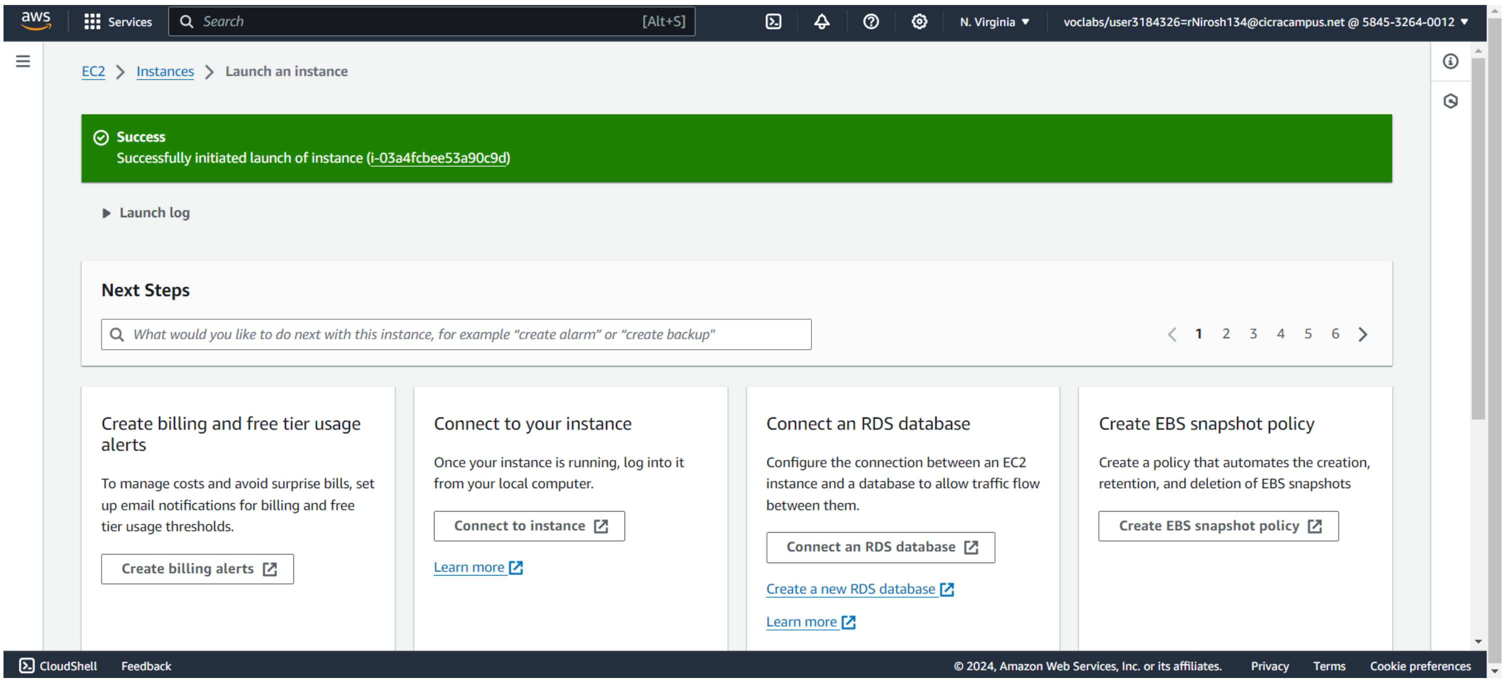


Task 7: Creating an EC2 instance in the private subnet

In this task, we're setting up a new EC2 instance in our Private Subnet. We'll configure it to allow SSH traffic from the bastion host. Also, we'll create a new key pair to access this instance.

First, we create a new key pair named 'vockey2' and download the appropriate file for our operating system.

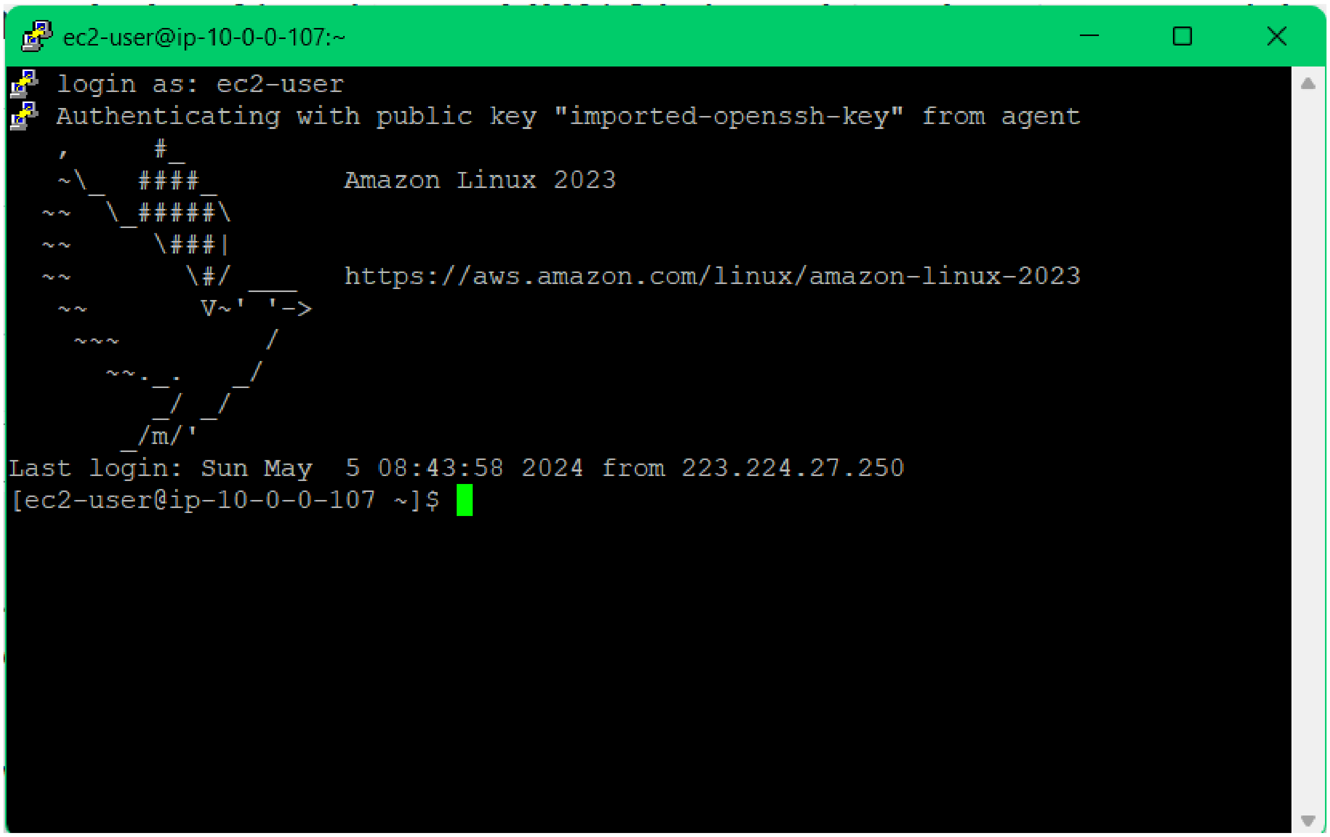
At last, we create the EC2 instance with the specified configurations



Task 8: Configuring your SSH client for SSH passthrough

To access the private instance securely without uploading the key pair to the bastion host, we'll use SSH passthrough. This allows us to use a key pair stored on our computer.

for that we use a software called Pageant from the PuTTY.



Task 9: Testing the SSH connection from the bastion host

In this task, we're testing the SSH connection from our bastion host to the EC2 instance in the Private Subnet.

First, we connect to the bastion host using SSH, using the method described in the SSH passthrough section.

Then, we connect to the private instance using SSH and its private IP address.

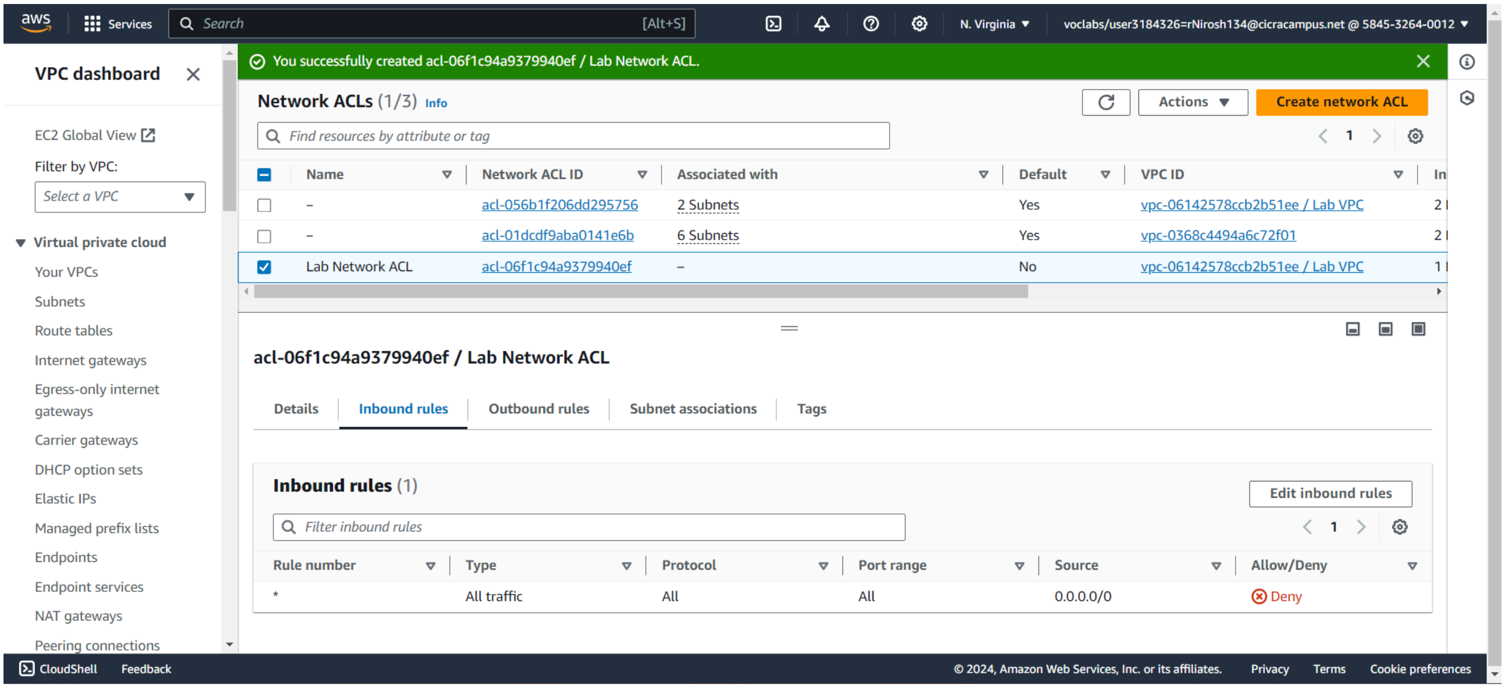
Once connected to the private instance, we test its internet connection by pinging 8.8.8.8.

This establishes communication between the Bastion Host in the Public Subnet and the EC2 instance in the Private Subnet

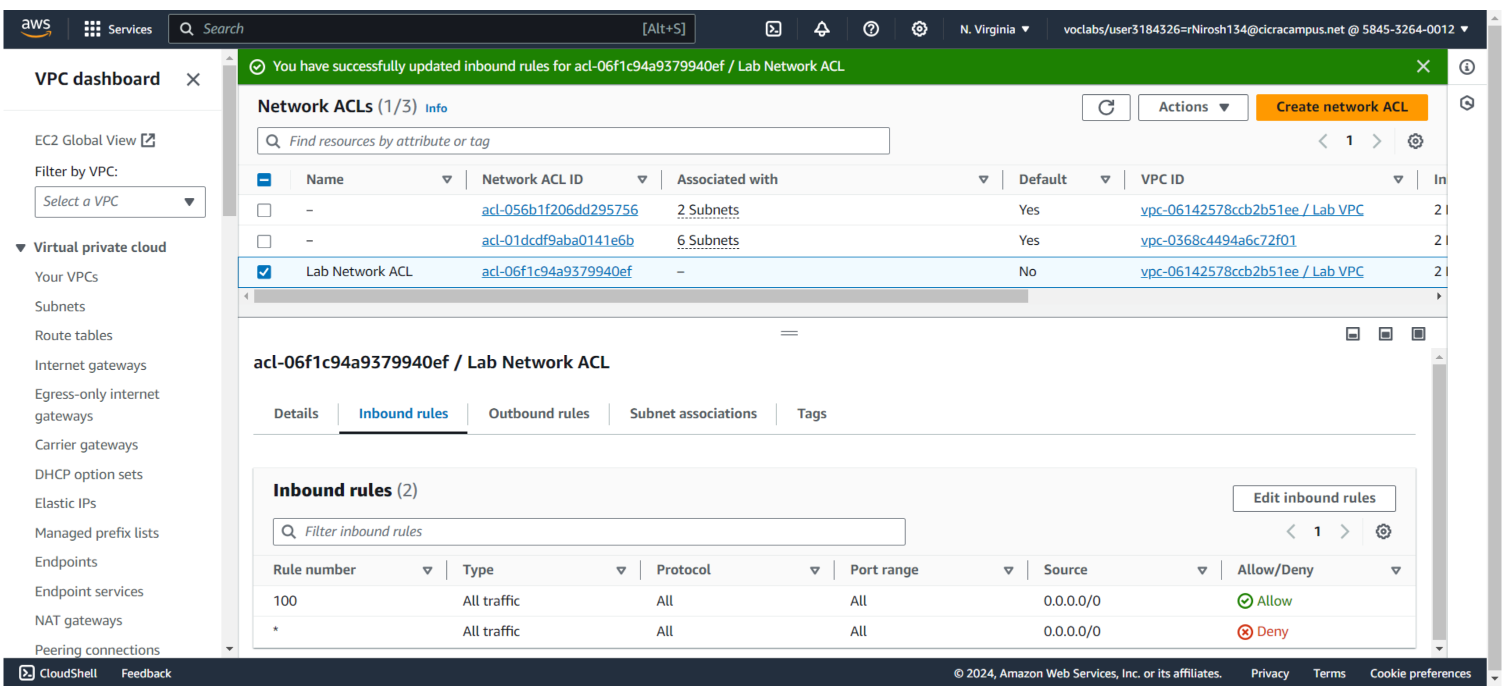
```
ec2-user@ip-10-0-1-160:~
The authenticity of host '10.0.1.160 (10.0.1.160)' can't be established.
ED25519 key fingerprint is SHA256:tAtBrAbOmT2CIcjoWiwZ3zxHMittEbkL848So8WVO1c.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.1.160' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-1-160 ~]$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=3.00 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=1.76 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=115 time=1.69 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=115 time=1.70 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=115 time=1.73 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=115 time=1.73 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=115 time=1.75 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=115 time=1.83 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=115 time=1.78 ms
^C
--- 8.8.8.8 ping statistics ---
9 packets transmitted, 9 received, 0% packet loss, time 8014ms
rtt min/avg/max/mdev = 1.687/1.884/2.995/0.394 ms
[ec2-user@ip-10-0-1-160 ~]$
```

Task 10: Creating a network ACL

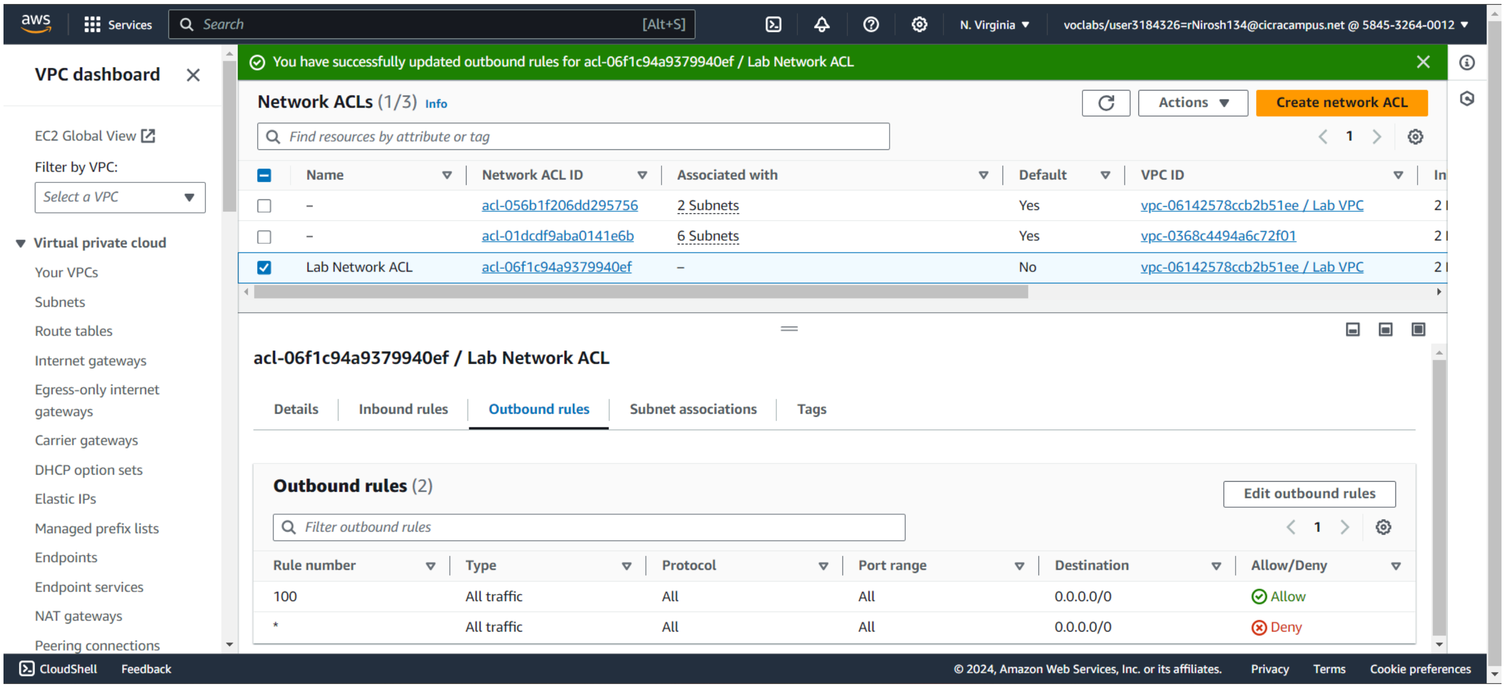
First, we'll create an EC2 instance in the Public Subnet and configure a security group to allow ICMP traffic from the local network.



Then, we'll create a custom network ACL called 'Lab Network ACL' for the Lab VPC. By default, this ACL will deny all traffic.

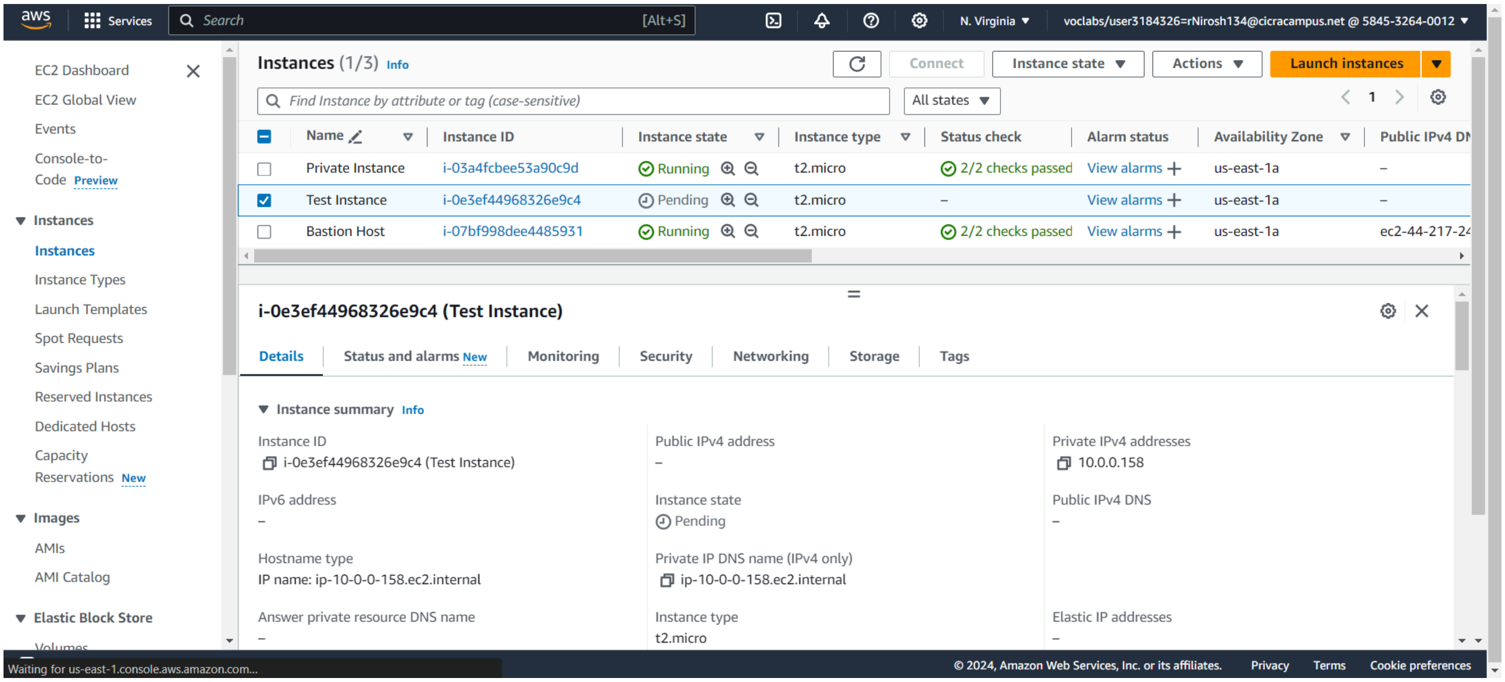


Finally, we'll configure the custom network ACL to allow all traffic into and out of the Private Subnet.

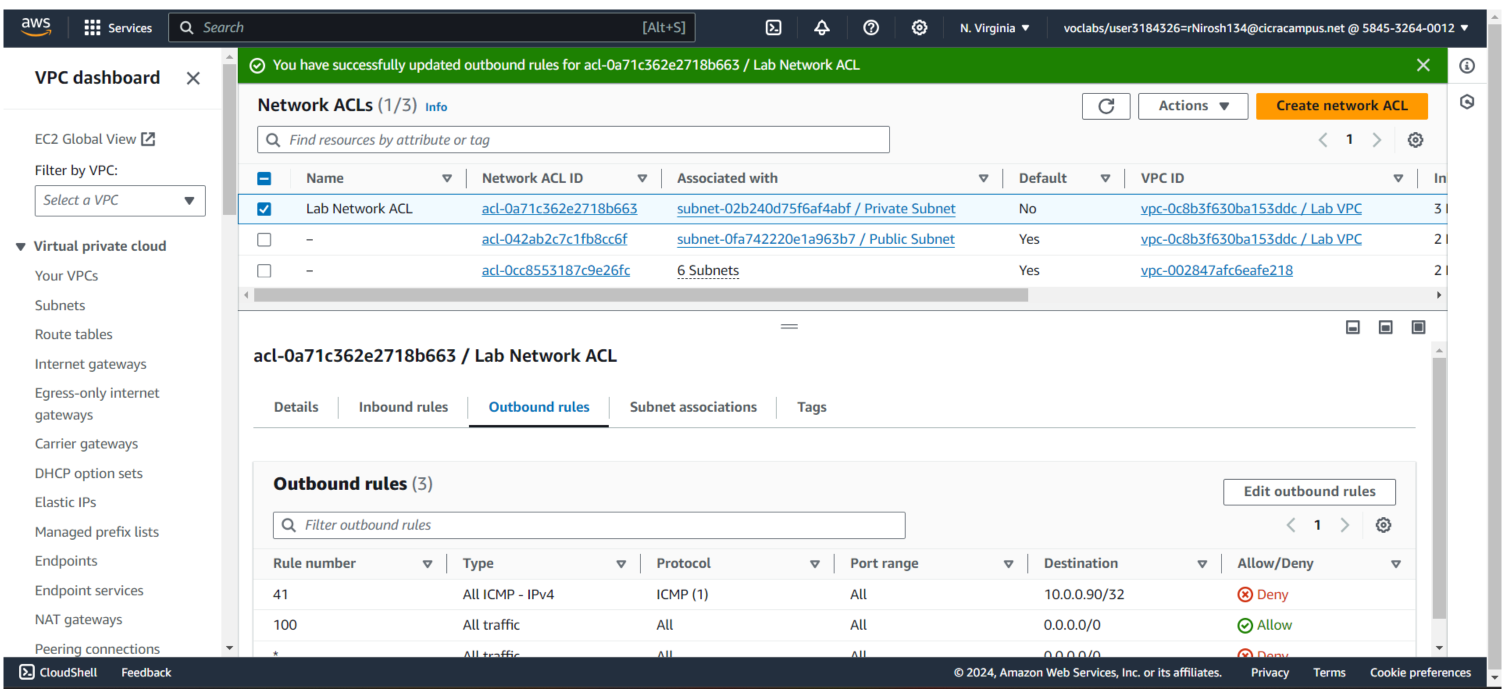


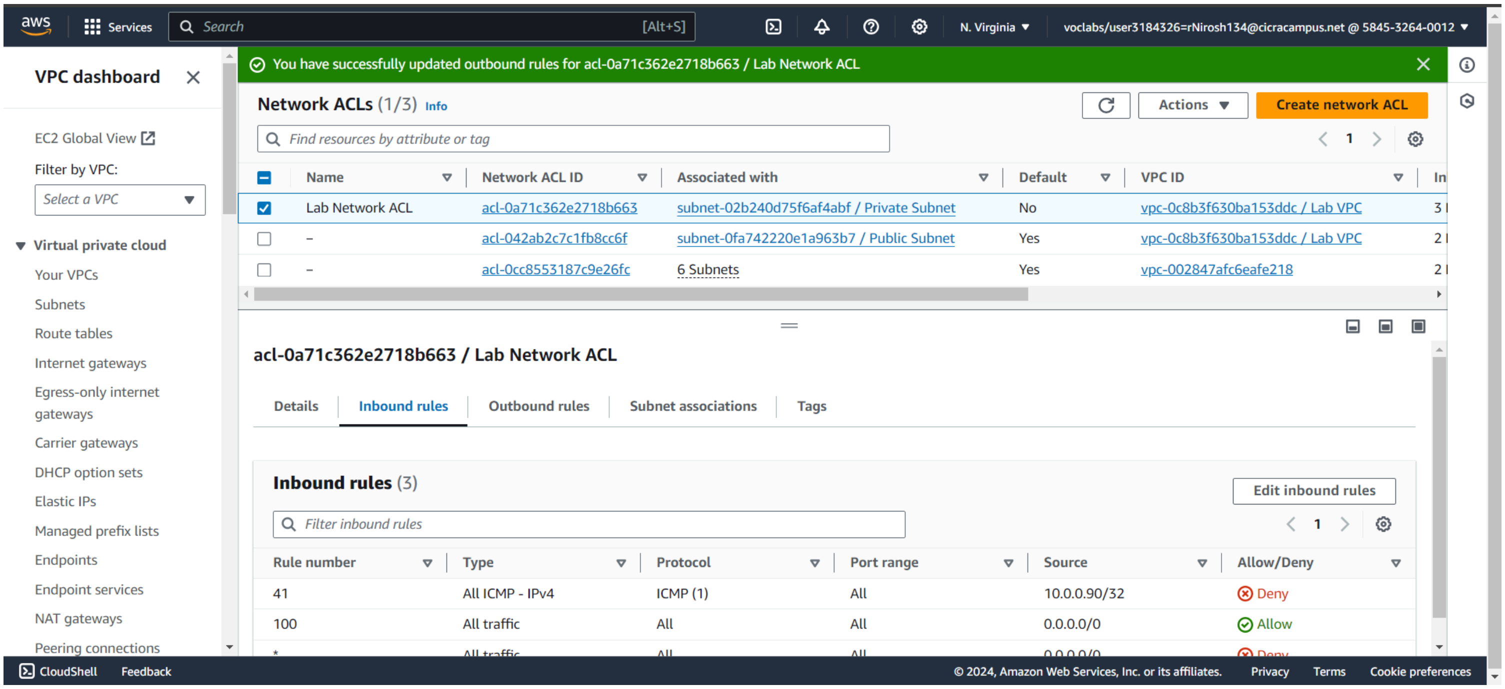
Task 11: Testing your custom network ACL

First we will create an instance in the EC2 called Test Instance in the public subnet.

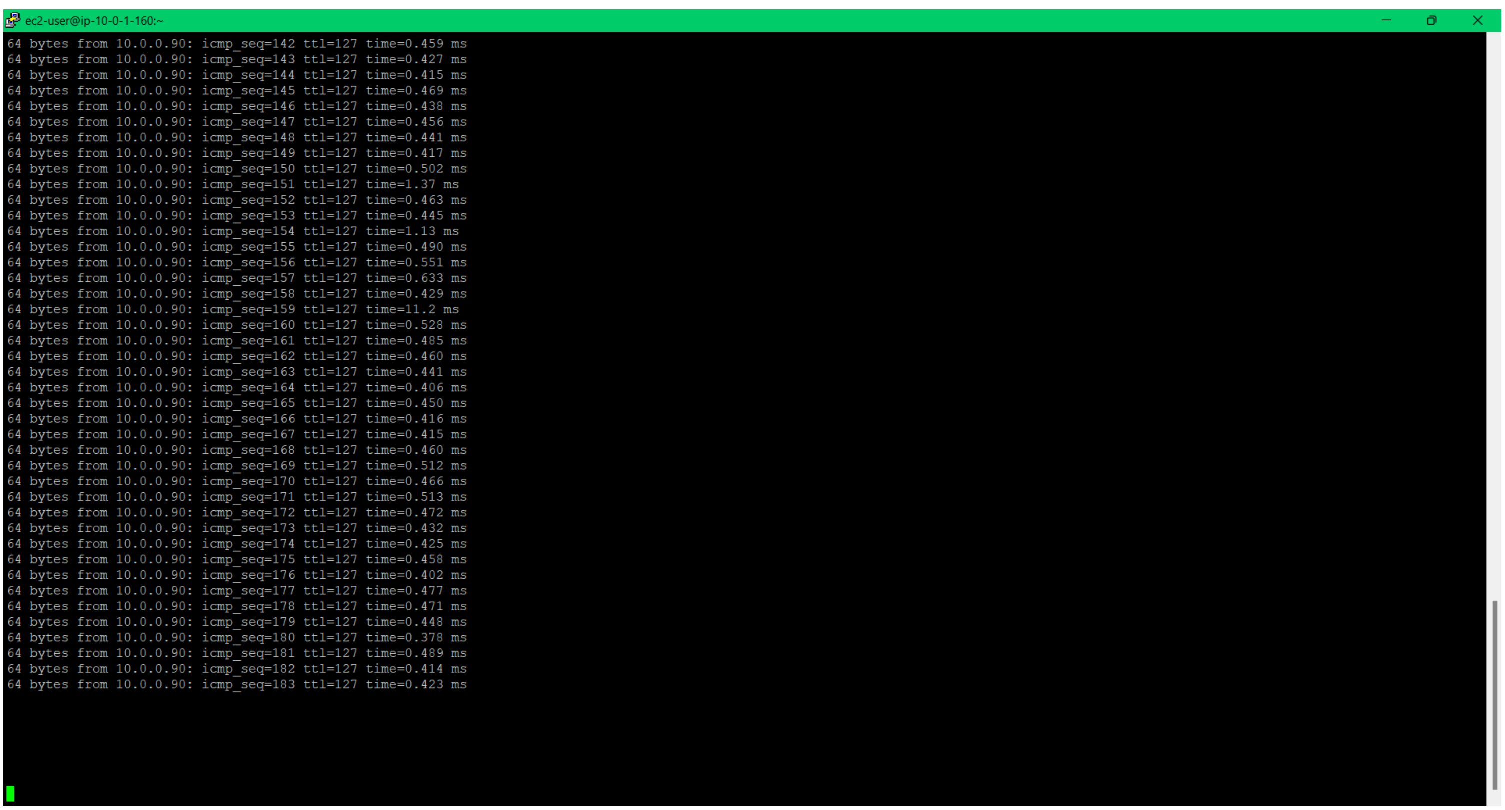


Then we will test the connection between the test instance and the private instance by pinging a message to the test instance and see if it replies.





Then we will set the ACL settings to stop the private instance from replying to the test instance, after applying this change, the ping message from the private instance won't get through to the Test instance anymore.



Grade:

Mt grade is 56/56